```
#include<iostream>
#include<stdlib.h>
#include<queue>
#include<omp.h>

using namespace std;

class node {
public:
    node* left, * right;
    int data;
};

class Breadthfs {
public:
    node* insert(node*, int);
    void bfs(node*);
};

node* Breadthfs::insert(node* root, int data) {
    if (!root) {
        root = new node;
        root->left = root->right = NULL;
        root->data = data;
        return root;
    }

    queue<node*> q;
    q.push(root);

    while (!q.empty()) {
        node* temp = q.front();
        q.pop();

        if (!temp->left) {
            temp->left = new node;
            temp->left->left = temp->left->right = NULL;
            temp->left->data = data;
            return root;
        } else {
            q.push(temp->left);
        }

        if (!temp->right) {
            temp->right = new node;
            temp->right->left = temp->right->right = NULL;
            temp->right->data = data;
            return root;
        } else {
            q.push(temp->right);
        }
    }
    return root;
}

void Breadthfs::bfs(node* head) {
    if (!head) return;
```

```
    queue<node*> q;
    q.push(head);

    while (!q.empty()) {
        int qSize = q.size();
        #pragma omp parallel for
        for (int i = 0; i < qSize; i++) {
            node* currNode;
            #pragma omp critical
            {
                currNode = q.front();
                q.pop();
                cout << "\t" << currNode->data;
            }

            #pragma omp critical
            {
                if (currNode->left) q.push(currNode->left);
                if (currNode->right) q.push(currNode->right);
            }
        }
    }
}

int main() {
    node* root = NULL;
    int data;
    char ans;
    Breadthfs tree;

    do {
        cout << "\nEnter data => ";
        cin >> data;
        root = tree.insert(root, data);

        cout << "Do you want to insert one more node? (y/n) ";
        cin >> ans;
    } while (ans == 'y' || ans == 'Y');

    cout << "\nBFS Traversal:\n";
    tree.bfs(root);

    return 0;
}
```

```
#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>
using namespace std;
const int MAX = 100000;
vector<int> graph[MAX];
bool visited[MAX];
void dfs(int node) {
  stack<int> s;
  s.push(node);
  while (!s.empty()) {
    int curr_node = s.top();
    s.pop();
    if (!visited[curr_node]) {
      visited[curr_node] = true;
      cout << curr_node << " ";
      #pragma omp parallel for
      for (int i = 0; i < graph[curr_node].size(); i++) {
        int adj_node = graph[curr_node][i];
        if (!visited[adj_node]) {
          #pragma omp critical
          s].push(adj_node);
        } } } }
int main() {
  int n, m, start_node;
  cout << "Enter No of Nodes, Edges, and Start Node: ";
  cin >> n >> m >> start_node;
  cout << "Enter Pairs of Edges: ";
  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    graph[u].push_back(v);
    graph[v].push_back(u);
  }
  #pragma omp parallel for
  for (int i = 0; i < n; i++) {
    visited[i] = false;
  }
  dfs(start_node);
  return 0; }
```

```
#include <iostream>
#include <omp.h>
using namespace std;
void parallelBubbleSort(int arr[], int n) {
    bool swapped;
    for (int i = 0; i < n - 1; ++i) {
        swapped = false;
          #pragma omp parallel for shared(arr, swapped) default(none)
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }
}
int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: \n";
    for (int i = 0; i < n; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    parallelBubbleSort(arr, n);
    cout << "Sorted array: \n";
    for (int i = 0; i < n; ++i) {
 cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

```
#include <iostream>
#include <omp.h>
using namespace std;
void merge(int arr[], int left, int mid, int right) {
  int n1 = mid - left + 1;
  int n2 = right - mid;
  int leftArr[n1], rightArr[n2];
  for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
  for (int j = 0; j < n2; j++) rightArr[j] = arr[mid + 1 + j];
  int i = 0, j = 0, k = left;
  while (i < n1 && j < n2) {
    if (leftArr[i] <= rightArr[j]) {
      arr[k] = leftArr[i];
      i++;
    } else {
      arr[k] = rightArr[j];
      j++;
    }
    k++;
  }
  while (i < n1) {
    arr[k] = leftArr[i];
    i++;
    k++;
  }
  while (j < n2) {
    arr[k] = rightArr[j];
    j++;
    k++;
  }
}
void parallelMergeSort(int arr[], int left, int right) {
  if (left < right) {
    int mid = left + (right - left) / 2;
    #pragma omp parallel sections {
      #pragma omp section
      parallelMergeSort(arr, left, mid);
      #pragma omp section
      parallelMergeSort(arr, mid + 1, right);
    }
    merge(arr, left, mid, right);
```

```
    }
}
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    omp_set_num_threads(4);
    parallelMergeSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

```
#include <iostream>
#include <omp.h>
using namespace std;
int main() {
   int n;
 cout << "Enter the number of elements: ";
   cin >> n;
    int arr[n];
   cout << "Enter the elements: ";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }
   int min_val = 2147483647;  // Maximum possible value for a 32-bit signed int
   int max_val = -2147483648; // Minimum possible value for a 32-bit signed int
   int sum = 0;
    #pragma omp parallel reduction(+:sum) reduction(min:min_val)
reduction(max:max_val)
   {
      #pragma omp for
      for (int i = 0; i < n; i++) {
         sum += arr[i];
         if (arr[i] < min_val) {
            min_val = arr[i];
         }
         if (arr[i] > max_val) {
            max_val = arr[i];
         }
      }
   }
 double average = static_cast<double>(sum) / n;
   cout << "Minimum value: " << min_val << endl;
   cout << "Maximum value: " << max_val << endl;
   cout << "Sum value: " << sum << endl;
   cout << "Average value: " << average << endl;
   return 0;
}
```

```
#include <iostream>
#include <cuda_runtime.h>

using namespace std;

__global__ void vectorAdd(int *A, int *B, int *C, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;

    if (index < n) {
        C[index] = A[index] + B[index];
    }
}

int main() {
    int *A, *B, *C;
    int *d_A, *d_B, *d_C;
    int N;

    cout << "Enter the size of the vectors: ";
    cin >> N;

    size_t size = N * sizeof(int);

    A = (int*)malloc(size);
    B = (int*)malloc(size);
    C = (int*)malloc(size);

    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    cout << "Enter the elements of the first vector (A):" << endl;
    for (int i = 0; i < N; i++) {
        cin >> A[i];
    }

    cout << "Enter the elements of the second vector (B):" << endl;
    for (int i = 0; i < N; i++) {
        cin >> B[i];
    }

    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

    cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);
```

```
    cout << "Result of vector addition (first 10 elements):" << endl;
    for (int i = 0; i < 10 && i < N; i++) {
        cout << C[i] << " ";
    }
    cout << endl;

    free(A);
    free(B);
    free(C);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

```
#include <iostream>
#include <cuda_runtime.h>
using namespace std;
__global__ void matrixMultiply(int* A, int* B, int* C, int width) {
   int row = blockIdx.y * blockDim.y + threadIdx.y;
   int col = blockIdx.x * blockDim.x + threadIdx.x;
   if (row < width && col < width) {
      int sum = 0;
      for (int k = 0; k < width; k++) {
         sum += A[row * width + k] * B[k * width + col];
      }
      C[row * width + col] = sum;
   }
}

int main() {
   int N;
   cout << "Enter matrix size (N x N): ";
   cin >> N;

   int *A, *B, *C;
   int *d_A, *d_B, *d_C;
   size_t size = N * N * sizeof(int);

   A = (int*)malloc(size);
   B = (int*)malloc(size);
   C = (int*)malloc(size);

   cudaMalloc((void**)&d_A, size);
   cudaMalloc((void**)&d_B, size);
   cudaMalloc((void**)&d_C, size);

   cout << "Enter elements of matrix A:" << endl;
   for (int i = 0; i < N * N; i++) {
      cin >> A[i];
   }

   cout << "Enter elements of matrix B:" << endl;
   for (int i = 0; i < N * N; i++) {
      cin >> B[i];
   }
```

```
    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks((N + 15) / 16, (N + 15) / 16);
    matrixMultiply<<<numBlocks, threadsPerBlock>>>(d_A, d_B, d_C, N);

    cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);

    cout << "Resultant matrix C:" << endl;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << C[i * N + j] << " ";
        }
        cout << endl;
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    free(A);
    free(B);
    free(C);

    return 0;
}
```